Study Unit 1: SDLC and Formal Methods

Expected Duration: 1 week

Introduction

Human develop from infanthood to childhood, then to adolescent, then adulthood. In each of these stages of human development, sickness can occur, however, steps need to be taken to minimize the occurrence of these sicknesses. In a similar manner, software development undergoes different stages, which occur in a cycle, called Software Development Life Cycle. Errors can occur in each of these stages of development. Use of mathematical methods, which is called formal methods in each stage of the SDLC minimizes these errors.

Learning Outcome for Study Unit 1

At the end of this study unit 1, you should be able to:

1.1 Define relevant keywords, which are related to SDLC and formal methods, (SAQ 1.1)
1.2 Identify and define the various stages of SDLC, (SAQ 1.2).
1.3 Identify and explain the use of formal methods at different stages of SDLC, (SAQ 1.3)

1.1 Different Stages of SDLC

Everything that you do in life consists of different stages, in the case of software development; it consists of different stages, which occurs in a life cycle, called Software Development Life Cycle. The various stages of SDLC are:

The SDLC can be divided into seven (7) stages;

1. Initial Study

This is the first time the system development team meets the clients to collective information regarding the problem. Normally this stage delivers the proposal and quotation to the clients.

2. Analysis

After the client has agreed to the proposal and price, the team will go in and study the current system with the intention to discover the source of the problem. The System analyst will use diagrams and data collection techniques (observation, inspections, interview, etc) to aid them. Normally this stage delivers a report stating the source of the problem and more than one alternative solutions.

3. Design

After the client agrees with the analysis findings, the client will choose one (1) solution. From this one solution the system designer will create the specification. Take note that different IT section will require different specification. For the software section, the deliverables will take the form of a screen design, logic design, representation of the codes, etc.

4. Development

Base on the given specification, the respective IT section will develop the solution. For the software section, the deliverables will be a full running software program created from the specification.

5. Testing

The test documents (Test Plan and Test Case) are normally created by the System Analyst during the development stages. The tester (normally a 3rd party) will use the Test Plan and Test Case to complete the testing. The deliverables will be a letter from the tester stating the outcome of the test.

6. Implementation

At this stage onward the software is no longer a concern, the main objective now will be to prepare the environment. The implementation plan will list the tasks necessary to prepare the environment to accept new software, such as installation, training, conversion of data, change over method, etc. There are many deliverables here depending on what is listed in the implantation plan. For example for user training a user manual is normally created.

7. Review

This is the final stage where the software user and team will sit down to review the software performance and to decide negotiate on the maintenance contract. If all goes well normally but not necessary a sign off letter will be the last deliverables.

1.2 Formal Methods at Different Stages of SDLC

At every stage of your life, you can always take precautionary measure to minimize the occurrence of sickness. Formal methods can be applied to each stage of the SDLC to minimize and eliminate errors during software development.

Formal method is a way to takes the specification (written in natural language) and converts it into its mathematical equivalent. Thus it is normally used in the SDLC Analysis and Design stages. The natural language usually contains ambiguous, incomplete and inconsistent statement. Once a specification in English for example is translated to a mathematical form, it will remove all ambiguity and uncertainty in that statement. Formal method will also bring to light all different probable perspective to any given variables and functions that could have

been hidden behind the English language. This can be done using a number of formal languages such as Z notation, VDM, Algebra, Functional Programming, etc.

In-Text Question 1

State the disadvantage of using natural language to specify software during the Analysis and Design stage of SDLC

1.3 Summary for Study Unit 1, Module 1

Having completed study unit 1 of module 1, you have learnt that:

1. SDLC and formal methods have relevant keywords and their meaning.
2. SDLC has different stages.
3. Formal methods can be used at different stages of SDLC.

1.4 Self Assessment Question for Unit 1, Module 1.

SAQ 1.1 (Tests Learning Outcome 1.1)

Define the term Analysis as a stage of SDLC

SAQ 1.2 (Tests Learning Outcome 1.2)

List the various stages of SDLC

SAQ 1.3 (Tests Learning Outcome 1.3)

Why is it important to use the formal method of formal specification during the Analysis and Design stages of SDLC.

Answer to In-Text Question 1

Using natural language to specify the function of a software can make the specification to be ambiguous, which can lead to error when the software is developed.

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 2: Advantages and Disadvantages of Formal Methods

Expected Duration: 1 week

Introduction

Life in general consists of the good and the bad. Everything we use in life has its advantage and disadvantage. The advantage emphasises the good aspect of the thing, while the disadvantage emphasises the bad aspect of the thing. The use of formal methods in the development of computer software has its advantages and its disadvantages as well.

Learning Outcome for Study Unit 3

At the end of this study unit 2, you should be able to:

1.1 Define the important keywords, which relates to the advantages and disadvantages of. They have been highlighted, (SAQ 1.1)
1.2 Identify and explain the various advantages of formal methods, (SAQ 1.2).
1.3 Identify and explain the various disadvantages of formal methods, (SAQ 1.3).

1.1 Advantages of Formal Methods

The good side of a thing is always desirable. The following are the advantages of formal methods:

Formal Method forces the **System Analyst** and **Designer** to think carefully about the **specification** as it enforce proper engineering approach using discrete mathematics. Formal Method forces the System Analyst and Designer to see all the different possible **states** for any given variables and functions thus will avoid many **faults** and therefore reduces the bugs and errors from the design stage onward

1.2 Disadvantages of Formal Methods

The bad side of a thing is always undesirable. The following are the disadvantages of formal methods:

Formal Method requires the person to know how to apply discrete mathematics. It will obviously slow down the analysis and design stage resources and time therefore also the cost of the project. There are too many different formal methods and most of them are not compatible with each other. Formal methods do not guarantee that a specification is complete. For each variable and function, it just forces the System Analyst and Designer to view the specification from a different perspectives but it does not guarantee that variable and functions will not be left out.

In-Text Question 1

State one advantage of formal methods.

1.3 Summary for Study Unit 2, Module 1

Having completed study unit 1 of module 1, you have learnt that:

1. The advantages and disadvantages of formal methods have important keywords, which has been highlighted.
2. Formal methods have advantages.
3. Formal methods have disadvantages.

1.4 Self Assessment Question for Unit 2, Module 1.

SAQ 1.1 (Tests Learning Outcome 1.1)

Define the term software specifications.

SAQ 1.2 (Tests Learning Outcome 1.2)

What are the advantages of formal methods.

SAQ 1.3 (Tests Learning Outcome 1.3)

What are the disadvantages of formal methods.

Answer to In-Text Question 1

Formal Method forces the System Analyst and Designer to see all the different possible **states** for any given variables and functions thus will avoid many **faults** and therefore reduces the bugs and errors from the design stage onward

---

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 3: Justifications for Formal Methods

Expected Duration: 1 week

Introduction

As human, everything that you do, you must have a reason for it. There are reasons why software engineers use formal methods during the development of computer software, The main reason is to prevent the failure of such system. Some systems are safety critical system; the failure of such system can lead to loss of life. The development of such systems requires the use of formal methods so as to achieve zero tolerance to failure. However, this thinking is now slowly fading as most clients realize the important and cost saving and convenience of having a good specification initially in the SDLC.

Learning Outcome for Study Unit 3

At the end of this study unit 4, you should be able to:

1.1 Define the important keywords that have been high lightened, which relate to the justifications for formal methods, (SAQ 1.1).
1.2 Identify and explain other critical systems that justify the use of formal methods, (SAQ 1.2).
1.3 Identify the various integrity levels when developing software, (SAQ 1.3)

1.1 Different Types of Critical Systems

There are basically three different types of critical systems;

1. Business Critical System:

Business Critical System refers to a system where the honesty and integrity of the business is paramount. All data kept in the system must be accurate at all times. If a fault is found the entire process must be stopped to allow correction. Most government, business and manufacturing company that require payment are business critical.

2. Mission Critical System

Mission Critical System refers to a system where the continuous running of the system is paramount. Accuracy takes a lower priority compared to the running of the system. Auto Teller Machine, Car ticketing system, Alarm Systems are mission critical.

3. Safety Critical System

Safety Critical System refers to a system where the safety of everyone directly or indirectly affected by the system is paramount. Functionality and Accuracy take a lower priority compared to the safety of the users. Most medical, construction and oil rig systems are safety critical system. Many organizations today require a combination of the above as such you may have a business mission critical system, a business safety critical system, etc

1.2 Integrity Levels

Since most critical systems are high risk systems, the integrity levels establish relationship between cost and risk. It defines how much cost is an organization is willing to spend and how much risk is an organization is willing to take when developing software.The Table below shows three integrity levels and the examples of systems that belong to each integrity level.

Integrity Level	Cost	Risk	Examples of Systems
1	Low	Low	Address Book System
2	Low	High	Global Tsunami Warning System
	High	Low	Waste Water System
3	High	High	Nuclear Reactor System

In-Text Question 1

Define the term safety critical system.

1.3 Summary for Study Unit 3, Module 1

Having completed study unit 3 of module 1, you have learnt that:

1. The justifications of formal method has important keywords, which has been highlighted.
2. The justification of formal methods involves three critical systems.
3. The integrity levels is defined based on cost and risk.

1.4 Self Assessment Question for Unit 3, Module 1.

SAQ 1.1 (Tests Learning Outcome 1.1)

Define the term integrity level.

SAQ 1.2 (Tests Learning Outcome 1.2)

List the various critical systems that justify the use of formal methods.

SAQ 1.3 (Tests Learning Outcome 1.3)

Use a table to show the various integrity levels and the cost and risk.

Answer to In-Text Question 1

Safety critical systems are systems that its failure can lead to loss of life or endanger the life of the user or people around.

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 4: The Theory of Formal Methods

Expected Duration: 1 week

Introduction

If you want to really understand a thing, try to look at the theory behind that thing. Good understanding of formal methods requires an understanding of some of the theory behind formal methods. Two theoretical concepts of formal methods will be considered in this unit, which include: Formal methods myths and some of the commandments of formal methods.

Learning Outcome for Study Unit 4

At the end of this study unit 4, you should be able to:

1.1 Define the important keywords that have been high lightened, which relate to the theory of formal methods (SAQ 1.1).
1.2 Identify and explain the various misconceptions about formal methods, (SAQ 1.2)

1.3 Identify and explain some of the rules and govern the use of formal methods, (SAQ 1.3).

1.1 Myths Related to Formal Methods

Sometimes when you want to do a particular thing, people can misunderstand you and create a wrong idea contrary to what you intended to do. This, sometimes is called misrepresentation or misconception. Based on the meaning of formal methods, as already stated earlier, some people have wrong ideas and misconception or false belief about formal methods. Below are some of the misconceptions or false beliefs or myths that people have about formal methods.

Myth 1: ***Formal Methods can guarantee that software is perfect.***
One should remember that *any* technique is fallible. Even if a correct mathematical proof *is* achieved, the assumption that the mathematics models reality correctly is still prone to error.

Myth 2: ***They work by proving that programs are correct.***
It is not *necessary* to undertake proofs to gain benefit from the use of formal methods; indeed much if not most industrial use of formal methods does not involve proofs. Major gains can be achieved just be formally *specifying* the system being designed since this process alone can expose flaws, and in a much more cost effectivemanner. Proofs may be worthwhile in highly critical systems where the extra cost can be justified.

Myth 3: ***Only highly critical systems benefit from their use.***
A range of formal methods have been applied to many types of system, some of greater, others of lesser criticality. The extent and type of application will depend on the level of criticality, which is ultimately a case of engineering and financial judgement.

Myth 4: ***They involve complex mathematics.***
The mathematics required (and desired!) for formal specification is of a level that could be taught at school. After all, a major goal of a specification is to be easily understandable, so using esoteric terminology is in nobody's interest. Unfortunately, although relatively simple, it is a fact that many software engineers have not received the requisite training in the past.

Myth 5: ***They increase the cost of development.***
Proofs *do* increase the cost of development in general, but formal *specifications* do not if used appropriately. This is because they allow many errors to be discovered earlier on in the design process when they are still relatively cheap to correct.

Myth 6: ***They are incomprehensible to clients.***
The mathematics may not be readable by an untrained client, but a formal specification helps produce a much clearer natural language description of the system as well. This should be presented to the client, giving a much less ambiguous description of the system than is often the case.

Myth 7: ***Nobody uses them for real projects.***
There are now a number of examples of actual use of formal methods, with demonstrably beneficial results. Two recommended examples which used Z, and both of which won UK Queen's Awards for Technological Achievement in 1990 and 1992, are the Inmos Transputer Floating Point Unit microcode design and the IBM CICS Transaction Processing System.

Myth 8: ***Formal methods delay the development process.***
Some projects using formal methods *have* been seriously delayed in the past, but this has been as much to do with the problem of introducing any new technique into the design process as to do with formal methods *per se*. The *over*-use of formal methods *does* delay the development process. Certainly full proofs are a time-consuming activity, which may not be (indeed, normally *will* not be) worthwhile. However the use of formal *specification* (e.g., on the IBM CICS project) and even formal development with appropriate personnel and tools (e.g., for the Inmos Transputer Floating Point Unit microcode) – see Myth 7 – as part of the development process have been demonstrated to be worthwhile, giving measurable improvements in cost and time.

Myth 9: ***They do not have tools.***
There are now some significant tools supporting formal methods, many of which have been put to serious industrial use. Large toolsets worthy of mention include the B-Tool, and the associated B-Toolkit from B-Core (UK) Limited, for the BMethod; the RAISE (Rigorous Approach to Industrial Software Engineering) development method, a more comprehensive successor to VDM, and its associated toolset available from CRI (Computer Resources International) in Denmark; the VDM Toolbox from IFAD, Denmark. Some theorem provers, such as EVES (based on ZF – Zermelo-Fraenkel – set theory), HOL (based on higher order logic), LP (the Larch Prover, for algebraic specifications), Nqthm (a successor to the Boyer-Moore theorem prover, from CLInc in Austin, Texas), OBJ and PVS (a more recent Prototype Verification System from SRI in California,based on higher order logic), have been used for significant proofs. Some can provide support for Z. A number of tools for Z are listed in page 225, together with relevant contact information.

Myth 10: ***Their use means forsaking traditional engineering design methods.***
Formal methods should *not* be used to replace the existing development process. Rather they should be slotted into the process in an appropriate and thoughtful manner. This can be a tricky issue which needs serious consideration by the project manager and all concerned. Considerable goodwill is required for this to be done in a smooth way. Method integration is an important issue, e.g., for structured and formal techniques. One example is the combination of SSADM and Z to produce SAZ. Formal methods can also be used effectly to augment an existing design process by providing extra feedback to correct errors early in the design process. *Cleanroom* is another approach, which could be combined with formal methods.

Myth 11: ***They only apply to software.***

Formal methods are used for hardware development as well as software. The Inmos Transputer work mentioned in Myth 7 is one example. Z has also been applied to microprocessor instruction sets, oscilloscopes, etc. For further examples, Hardware/software co-design is a rapidly developing area and formal methods could be useful in clarifying this difficult area.

Myth 12: ***They are not required.***
Increasingly, standards *will* mandate, or at least highly recommend, the use of formal methods for systems of the highest integrity, such as those that are safety critical.

Myth 13: ***They are not supported.***
There are now many books on formal methods (including this one!), conferences, courses, etc. For pointers specifically concerned with Z, see Appendix A. A number of companies now specialize in formal methods (e.g., B-Core (UK) Limited, Computational Logic Inc., CRI, DST, Formal Systems (Europe) Limited, IFAD, Praxis). A range of formal methods tools are commercially marketed (e.g., the FDR model checker from Formal Systems, the LAMBDA toolset from Abstract Hardware Limited, Proof Power by ICL, etc.).

Myth 14: ***Formal methods people always use formal methods.***
While formal methods *can* be useful, they are not always appropriate. Even those well versed in the use of formal methods do not always use them. This can be particularly useful to communicate design ideas within a team. Thus in a design team of one for a small project they may not be worthwhile.

In-Text Question 1

What does the formal methods myths means?

1.2 Commandments of Formal Methods

If you are a student in a school or an employee in an organisation, there are always rule and regulation and guide your activity on the school or the organisation. These rules and regulations ensure that everything is done in a way that is expected. The use of formal methods in software development has its rules and regulations, which has been termed in some literature as the commandments of formal methods. They are as follows:

1st commandment: *Thou shalt choose an appropriate notation.*
Z is appropriate if you wish to undertake formal specification as part of a design team. Other formal notations and methods have different strengths and weaknesses depending on what is required of the method in the development process. For example, the B-Method is more

suitable than Z if formal development with tool support is to be undertaken. Many factors will affect the selection of a notation, not least of which is the background and expertise of the team involved. Learning a new notation is a time-consuming process which should be avoided if an appropriate (enough) notation is already know by the team.

2nd commandment: *Thou shalt formalize but not over-formalize.*
This relates to Myth 14. In addition, getting the right level of abstraction is very important in a specification. This will affect its readability, the ease with which proofs can be undertaken, and the number of design choices left open for the developer. The level of abstraction should be as high as possible, but no higher; otherwise important information may be omitted.

3rd commandment: *Thou shalt estimate costs.*
This is perhaps one of the most difficult things to do for any software product, whatever the development approach. Unfortunately the estimation technique used in many other engineering disciplines break down for software. The complexity of the solution is very difficult to estimate before it has been undertaken. Long experience can help to give some insight, but estimates are still difficult. Formal methods may help to quantify the estimation, since a formal description of the problem is obtained early in the design process. However, the effort to produce the final implementation may be difficult to determine from the formal specification.

4th commandment: *Thou shalt have a formal methods guru on call.*
Formal methods *do* require significant mathematical ability and training. These are not beyond the level obtainable by the average engineer, but it is still worthwhile having an expert with several year's experience of formal methods available, at least on an easily accessible consultancy basis, especially if many of the design team are relatively new to formal methods. This could well avoid a great deal of unnecessarily wasted time and cost.

5th commandment: *Thou shalt not abandon thy traditional development methods.*
This relates to Myth 10. Formal methods should be used as an extra technique in the armoury available for the elimination of errors. Certainly they will not catch all the errors, so other techniques should also be used (e.g., see the 9th commandment below).

6th commandment: *Thou shalt document sufficiently.*
Much of this book is dedicated to this subject, hopefully demonstrating that a formal notation like Z can be used in a beneficial way for system documentation. Even if the formal specification is omitted from the final documentation, its production is likely to make the informal documentation clearer and less ambiguous.

7th commandment: *Thou shalt not compromise thy quality standards.*
Software quality standards such as ISO 9000 need to be met, whatever the development techniques that are used. Formal methods can help in this respect if applied sensibly, but the project manager should ensure that they do help rather than hinder in practice.

8th commandment: *Thou shalt not be dogmatic.*
Absolute correctness in the real world can never be achieved. Mathematical models can be verified with a good level of certainty, but these models might not correspond with reality correctly. When applying formal methods, the level of use should always be determined beforehand and monitored while in progress. A project manager should always be prepared to adjust the level of use if required.

9th commandment: *Thou shalt test, test, and test again.*
Formal methods will never replace testing; rather they will reduce the number of errors found through testing. Formal development and testing tend to avoid and discover different types of error, so the two are complementary to some extent.

10th commandment: *Thou shalt reuse.*
Formal specifications can be written in a reusable manner, with some thought. As an example, Z includes a 'toolkit' of definitions, defined in Z itself, which have proved to be useful for many different specifications. The core of the toolkit is accepted as standard by most people who use Z for specification. In this book, the Common Service Framework mentioned, the machine word definitions and the graphics definitions, could all be reused – indeed, *have* been reused – for other specifications. The above 'commandments' will hopeful provide basic guidance in the use of formal methods in practice.

Answer to In-Text Question 1

Formal methods myths are the wrong ideals, and concepts and belief about formal methods in software development.

1.3 Summary of Study Unit 4

Having completed study unit 4 of module 1, you have learnt that:

1. There are important keywords that are related to the theory of formal methods.
2. There are various formal methods myths, which are the wrong beliefs about formal methods in software development.
3. For effective use of formal methods in software development, there are various rules that govern its usage.

1.4 Self Assessment Question for Study Unit 4

SAQ 1.1 (Tests Learning Outcome 1.1)

What do you understand by the term, formal methods commandments?

SAQ 1.2 (Tests Learning Outcome 1.2)

State the second myth of formal method.

SAQ 1.3 (Test Learning Outcome 1.3)

State the third commandment of formal methods.

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 5: Classifications of Formal Methods

Expected Duration: 1 week

Introduction

In anything that you do in life, including the development of computer software, error can occur at any time. In software development, in order to prevent these errors from occurring, formal methods can be used at different stages of software development. The uses of formal methods at different stages of software development lead to the different classes of formal methods.

Learning Outcome for Study Unit 5

At the end of this study unit 5, you should be able to:

1.1 Define the important keywords, which relates to the various classes of formal methods. They have been highlighted, (SAQ 1.1)
1.2 Explain formal specification as a class of formal methods, (SAQ 1.2).
1.3 Explain formal proof as a class of formal methods, (SAQ 1.3).
1.4 Explain model checking and abstractions as classes of formal methods, (SAQ 1.4).

1.1 Formal Specification

Every product that you use was produced to perform a specific function. Computer software are developed through the communication between the customer and the developer. The customer communicates with the developer specifying what the software will do. However, using English language or natural language to specify what the system will do may be very ambiguous; therefore there is the need to represent the specification of the system using

mathematical language, for precise and more accurate specification. This is called formal specification.

This is where normal system specification is use and translated using a formal language into a formal specification. There are basically two type of formal language; Model Oriented (VDM, Z, etc) and Properties Oriented (Algebraic Logic, Temporal Logic, etc). This is the cheapest way to handle formal method. The formal specification generally does the following process.

1. Get user requirement usually from the specification written in the natural language.

2. Clarify the requirement using mathematical approach. This is to remove all ambiguous, incomplete and inconsistent statement.

3. After statements are clearly identified. Then find all assumptions (Things that must be in place before something can happen) that is state or not stated within the clarified requirement.

4. Then expose every possible logic defect (fault) or omission in the clarified requirement.

5. Identify what are the exceptions (bad things) that will arise if the defects are not corrected.

6. Find a way to test for all the possible each exception. Only when you can test for an exception can you be able to stop that exception from happening.

1.2 Formal Proof

After you have produced any product, there will be the need to verify that the product actually performed the intended function. After the development of computer software, there will be the need to verify that the software performs its specified functions. This verification is done using mathematical proof, called formal proof. This level studies the formal specification and retrieves the goals of the formal specific. Then fixed rules are created and with these rules step by step instructions are listed to achieve the specified goals. This is relatively cheaper but there are more task steps.

1.3 Model Checking

Whenever you have finished a thing, it is always good for you go check through in order to make sure that what you have done is correct. This level studies the formal specification and formal proof deliverables to make sure that the system or software contains ALL possible properties to be able to handle all possible scenarios that could happen for a given specification. This stage is beginning to be more expensive.

1.4 Abstraction/Mathematical Modelling

Everything you do in life can be represented as a mathematical model. This level uses mathematical and physical models to create a prototype of the entire system for simulation.

This prototype is use to focus on the properties and characteristic of the system. This is the most expensive formal method.

In-Text Question 1

List the various types of formal methods.

1.5 Summary for Study Unit 2, Module 1

Having completed study unit 5 of module 1, you have learnt that:

1. The classifications of formal methods have important keywords, which has been highlighted.
2. Formal specification is a class/type of formal methods.
3. Formal proof is a class/type of formal methods.
4. Model checking and Abstraction/Mathematical modelling are also types of formal methods.

1.6 Self Assessment Question for Unit 5, Module 1.

SAQ 1.1 (Tests Learning Outcome 1.1)

Define the term formal specifications.

SAQ 1.2 (Tests Learning Outcome 1.2)

List the various classes/types of formal methods.

SAQ 1.3 (Tests Learning Outcome 1.3)

Distinguish between formal specification and formal proof.

SAQ 1.4 (Tests Learning Outcome 1.4)

The class/type of formal method, which builds a prototype that is used to focus on the properties and characteristics of the system is called .....

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011

2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Notes for Self Assessment Questions (SAQs) for Study Unit 1.

SAQ 1.1

This is one of the stages of SDLC where the team will go in and study the current system with the intention to discover the source of the problem. The System analyst will use diagrams and data collection techniques (observation, inspections, interview, etc) to aid them.

SAQ 1.2 (Tests Learning Outcome 1.2)

1 **Initial Study**
2 **Analysis**
3 **Design**
4 **Development**
5 **Testing**
6 **Implementation**
7 **Review**

SAQ 1.3 (Tests Learning Outcome 1.3)

Because using English language or natural language to describe the system is ambiguous and may not lead to precise description of the system, unlike the use of mathematical specification language, which gives precise and accurate description of the system.

Notes for Self Assessment Questions (SAQs) for Study Unit 2.

SAQ 1.1

Software specification is a term that means description of what the system will do.

SAQ 1.2

It help to minimize and eliminate errors during the development of software, especially for critical systems.

SAQ 1.3 (Tests Learning Outcome 1.3)

Additional time and cost will be required in applying formal methods during software development.

Notes for Self Assessment Questions (SAQs) for Study Unit 3.

SAQ 1.1

It defines how much cost is an organization is willing to spend and how much risk is an organization is willing to take when developing software.

SAQ 1.2

Business Critical System:

Mission Critical System

Safety Critical System

SAQ 1.3

Integrity Level	**Cost**	**Risk**	**Examples of Systems**
1	Low	Low	Address Book System
2	Low	High	Global Tsunami Warning System
	High	Low	Waste Water System
3	High	High	Nuclear Reactor System

Notes for Self Assessment Questions (SAQs) for Study Unit 4.

SAQ 1.1

Formal methods commandments are the rule that guide the use of formal methods during software development, for effective results.

SAQ 1.2

They work by proving that programs are correct.

SAQ 1.3

Thou shalt estimate costs.

Notes for Self Assessment Questions (SAQs) for Study Unit 5.

SAQ 1.1 (Tests Learning Outcome 1.1)

Formal specification is the use of mathematical language to state the functionalities of computer software.

SAQ 1.2 (Tests Learning Outcome 1.2)

Formal specification

Formal proof

Model checking

Abstraction/Mathematical modelling

SAQ 1.3 (Tests Learning Outcome 1.3)

Formal specification uses mathematical language to describe the functional of the software, while formal proof uses mathematical language to prove that the software performs its specified function.

SAQ 1.4 (Tests Learning Outcome 1.4)

.....Abstraction/Mathematical modelling.....

Module 2: Mathematical Tools Used in Formal Methods

Overview:

If you are a professional, in a particular area, then you will have some set of tools that you need to do your work. Software engineers, who apply formal methods to software development use set of mathematical tools. This module identifies and discusses the various mathematical tools that formal methods use. It consists of the following study units:

Study unit 1, Proposition

Study unit 2, Predicates

Study unit 3, Sets

Study unit 4, Mathematical proofs

Study Unit 1: Proposition

Expected Duration: 1 week

Introduction

Every statement that you make can either be true of false. These statements that results in either true or false are called propositions. A proposition can be represented using an alphabet. In formal methods, the natural language is scan for propositions. Each proposition (either or false) will be translated into an expression usually joined using operators, and all the possible value for each proposition will be listed in a truth table to cover all possible value. This unit introduces proposition as a mathematical tool that can be used to describe and prove the correctness of a system.

Learning Outcome for Study Unit 1

At the end of this study unit 1 of module 2, you should be able to:

1.1 Explain the meaning of important keywords, which have been highlighted and are related to proposition, (SAQ 1.1).
1.2 Identify and state the meaning of the different connectives of proposition, (SAQ 1.2).
1.3 Use truth table to evaluate complex proposition, (SAQ 1.2).

1.1 Different Connectives of Proposition.

Most of the things that you use in life consists of small parts, which are connected together to form a whole. A proposition can be connected to another proposition using different connectives of propositions. These connectives can also be called operators and they are represented by different symbol. Each operator or connective of proposition has its own meaning. Table 1 below shows the various connectives or operators of proposition.

Precedence	Symbol	Meaning	Natural Language Example
1	()		
2	⌐ ~	NOT	Fail means NOT Pass
3	^	AND Conjunction	Hard work AND good attitude
4	v	OR Disjunction	Code in Java OR Code in C++
5	→	Conditional Implies	If you pass then you get reward
6	↔	Equals Bi-directional Bi-implication	Pass if and only if marks above 40
	⊗	Different Exclusive	Success is different from Failure

1.2 Meaning of the Different Connectives/Operation.

The understanding of anything that you are doing lies in the meaning of that thing. To understand the usage of the various connectives of proposition, you need to know the meaning of each of them. You can use a truth table to understand the meaning of each of the connectives/operations of proposition. A truth table shows all the possible combinations of variables of proposition and the result of each of the combinations. The following examples illustrate the meaning of each of the connectives of propositions.

1 The NOT(⌐) Connective/Operation

Suppose the proposition, A = 'Ali is a boy'. The truth table below shows all the possible values of the NOT connective/operation and the result.

A = Ali is a boy	⌐A = Ali is NOT a boy
T	F
F	T

2. The AND, Conjunction, ^ Connective/Operation

Consider these two propositions, A = Ali is a boy, M = Mary is a girl. These two propositions can be combined into one proposition using the AND/Conjunction/^ connective/operation, in the truth table below:

A = Ali is a boy	M = Mary is a girl	A^M = Ali is a boy AND Mary is a girl

T	T	T
T	F	F
F	T	F
F	F	F

3. The OR, Disjunction, v Connective/Operation

Consider these two propositions, A = Ali is a boy, M = Mary is a girl. These two propositions can be combined into one proposition using the OR/Disjunction/^ connective/operation, in the truth table below:

A = Ali is a boy	**M = Mary is a girl**	**A∨M = Ali is a boy OR Mary is a girl**
T	T	T
T	F	T
F	T	T
F	F	F

4. The Conditional/Implies/→ Connective/Operation

Consider these two propositions, A = Ali is a boy, M = Mary is a girl. These two propositions can be combined into one proposition using the Conditional/Implies/→ connective/operation, in the truth table below:

A = Ali is a boy	**M = Mary is a girl**	**A→M = Ali is a boy → Mary is a girl**
T	T	T
T	F	F
F	T	T
F	F	T

5. The Equals/Bi-directional/Bi-implication/↔ Connective/Operation

Consider these two propositions, A = Ali is a boy, M = Mary is a girl. These two propositions can be combined into one proposition using the Equals/Bi-directional/Bi-implication/↔ connective/operation, in the truth table below:

A = Ali is a boy	**M = Mary is a girl**	**A↔M = Ali is a boy ↔ Mary is a girl**
T	T	T
T	F	F
F	T	F
F	F	T

6. The Different/Exclusive/⊗ Connective/Operation

Consider these two propositions, A = Ali is a boy, M = Mary is a girl. These two propositions can be combined into one proposition using the Conditional/Implies/⊗ connective/operation, in the truth table below:

A = Ali is a boy	**M = Mary is a girl**	**A⊗M = Ali is a boy ⊗ Mary is a girl**
T	T	F

T	F	T
F	T	T
F	F	F

In-Text Question 1

Suppose A and B are two propositions, use a truth table to state the meaning of this connective/operation, A ⊗ B.

1.3 Evaluating Complex Propositional Statement Using Truth Table.

Anything that you call complex can be broken into many simple parts, thereby making it easy to handle the complex thing. A complex propositional statement is a propositional statement, which consists of more than one connective/operator. The evaluation of such complex propositional statement can be handled by breaking it into many simple propositional parts. The following example illustrates further:
Suppose P and Q are two propositions, use a truth table to evaluate this complex proposition, $(P \rightarrow Q) \vee (Q \rightarrow P)$

P	Q	$P \rightarrow Q$	$Q \rightarrow P$	$(P \rightarrow Q) \vee (Q \rightarrow P)$
T	T	T	T	T
T	F	F	T	T
F	T	T	F	T
F	F	T	T	T

The result of the complex proposition is TRUE for all the values of the P and Q, therefore, this complex propositional statement is called **TAUTOLOGY/VALID**. However, when all the results is FALSE for all the values of P and Q, then the proposition is called **UNSATISFIABLE/INVALID**. The proposition is called **SATISFIABLE** when at least one truth value brings a true result. It is called **CONTINGENT** when at least one truth value gives a true result and at least one false value give a true result

---

A = Ali is a boy	**B = Mary is a girl**	**A⊗B = Ali is a boy ⊗ Mary is a girl**
T	T	F
T	F	T
F	T	T
F	F	F

1.4 Summary of Study Unit 1 of module 2

Having completed study unit 1 of module 2, you have learnt that:

1. There are important keywords that are related to proposition, which have been highlighted.

2. Proposition has different connectives/operators, and their meaning can be understood using truth table.
3. Complex propositions have more than one connectives/operators and they can also be evaluated using truth table.

1.5 Self Assessment Question for Study Unit 1 of Module 2.

SAQ 1.1 (Tests Learning Outcome 1.1)

Define the term Contingent?

SAQ 1.2 (Tests Learning Outcome 1.2)

What name can you use to describe this proposition, ⌐ P ^ ⌐Q

SAQ 1.3 (Tests Learning Outcome 1.3)

Use a truth table to evaluate this complex propositional statement, .P→ (Q → R)

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 2: Predicates

Expected Duration: 1 week

Introduction

When you use a tool to perform a specific function, you may not use it to perform another function, in that case the limitations of one tool, calls for the need for another tool. Proposition is ideal when the constant declaration does not change. However, this is not practical as most statement actually contains variables and changes in the variables will change the validity of the statement to true or false, because some statement refers to a set of different elements. Therefore you need predicate to handle such statement. For this subject, First-Order Predicate Logic will be used.

Learning Outcome for Study Unit 2 of Module 2.

At the end of this study unit 2, you should be able to:

1.1 Explain the meaning of important keywords that relate to first order predicate logic, which have been highlighted, (SAQ 1.1).
1.2 Identify and explain the elements of a first order predicate logic, (SAQ 1.2).
1.3 Explain the use of Quantifiers in first order predicate logic, (SAQ 1.3).
1.4 Explain the use of Free and Bound Variables, (SAQ 1.4).

1.1 Basic Elements of First Order Predicate Logic

Everything that you use has different elements, which are used to form that thing. First order predicate logic is made of the following basic elements:

- Variables/constants: Elements that change or do not change in the statement.
- A function that performs the variable: Action that is performed on the constant/variable.
- Result: This can be true or false
- Connectives/Operators: These are the various connectives/operators of first order predicate logic. They are the same as the operators of proposition.

The following example illustrates how these elements can be identified in a natural language description.

An ostrich has wing can fly, an eagle has wing can fly.

The constant object here is “Wing” The variable object here is either “Ostrich” or “Eagle” The function here is “Fly”.

These elements can be tabulated in the table below:

Constant/Variable	**Function**	**Result**
Wing, Ostrich	Fly	False
Wing, Eagle	Fly	True

The connectives/operators of first order predicate logic are the same operators as proposition, with the same meaning. They include the following:

Predicate Example	**Symbol**	**Meaning**
⌐ Fly (Airplane)	⌐	Airplane cannot fly
Fly (Airplane) ^ Fly (Birds)	^	Airplane fly AND Bird fly
Fly (Airplane) v Fly (Birds)	V	Airplane fly OR Bird fly
Repair (Airplane) → Fly (Airplane)	→	If airplane is repaired then airplane will fly.
Repair (Car) ↔Repair(Bus)	↔	Repair the Car is the same as Repair Bus
Repair (Airplane) ⊗Repair(Bus)	⊗	Repair the airplane is different from Repair Bus

1.2 Quantifiers in First Order Predicate Logic

Sometimes, you need a notation to specify the number of items that a particular thing is made of. In first order predicate logic, Quantifiers are part of the first order predicate logic that specify the size of constant/variable. There are two types of quantifiers in first order predicate logic, they are presented in the table below:

Quantifier Symbol	**Quantifier Name**	**Quantifier Example**

∃	Existential	There exists some or For some the elements including itself
∀	Universal	For every element For all the elements obviously including itsel

From the above table, Existential quantifiers are used to one or some of the constant/variables, while Universal quantifiers are used to specify all the constants/variables.

Illustrating with these two statements: “Some airplane can fly” and “All airplanes can fly”. In these two statements, the constant/variable is airplane, the function is fly, therefore, they can be represented in first order predicate logic using the quantifiers as follows:

∀Airplane Fly (Airplane) = All airplane can fly .
∃Airplane Fly (Airplane) = Some airplane can fly.
Furthermore, if you have two sets of Airplane (Plane A and Plane B) using the above “Fly Faster” function you can have four (4) different statements. Plane A is denoted as A and Plane B is denoted as B.

- ∀A ∀B Fly Faster (A, B) = All Plane A fly faster than All Plane B
- ∀A ∃B Fly Faster (A, B) = All Plane A fly faster than Some Plane B
- ∃A ∀B Fly Faster (A, B) = Some Plane A fly faster than All Plane B
- ∃A ∃B Fly Faster (A, B) = Some Plane A fly faster than Some Plane B

In-Text Question 1.1

Suppose you have two sets of different object Boys and Girls using the “Run Faster” function, write four different first order predicate logic using the relevant quantifiers and their corresponding English equivalent.

1.3 Free and Bound Variables

When you bound a criminal in a prison, he/she is restricted in movement; on the other hand, the movement of a free man is not restricted. In a similar manner, the variables that you use in first order predicate logic can be free or bound. It is bound if the variable is applied to the quantifier; it is free if the variable is not applied to the quantifier. It is neither bound nor free if the variable does not occur in the first order predicate logic. The following examples illustrate further.

1. ∀x, function (x, y)

2. ∃x, function (x, y)

In the above examples of first order predicate logic, variable x is bound variable because it is applied to the quantifiers, while variable y is free because it is not applied to the quantifiers. Variable w is neither bound nor free because it is not used in the first order predicate logic.

Answer to In-Text Question 1.1

Suppose Boys is denoted as B and Girls is denoted as G.

∀B ∀G Run Faster (B, G) = All Boys run faster than All Girls

∀B ∃G Run Faster (B, G) = All Boys run faster than Some Girls

∃B ∀G Run Faster (B, G) = Some Boys run faster than All Girls

∃B ∃G Run Faster (B, G) = Some Boys run faster than Some Girls

---

1.4 Summary of Study Unit 2

Having completed study unit 2, you have learnt that:

1. Predicates have important keywords and their various definitions.
2. There are basic elements that make up first order Predicate logic.
3. First order predicate logic has two basic Quantifiers.
4. There are different types of variables in first order predicate logic.

1.5 Self Assessment Question for Study Unit 2 of Module 2

SAQ 1.1 (tests Learning Outcome 1.1)

Define the term Quantifiers

SAQ 1.2 (Tests Learning Outcome 1.2)

List the various elements of first order predicate logic.

SAQ 1.3 (Tests Learning Outcome 1.3)

List and explain the two Quantifiers in first order predicate logic.

SAQ 1.4 (Tests Learning Outcome 1.4)

1. Distinguish between free variable and bound variable.
2. Assuming the following: A is an animal, B is a bear, the function IsA(B,A) means that bear is an animal. Show all possible quantifiers for the above and state if that predicate statement is true or false and explain why.

   Answer:

∀A∀B IsA(B, A) = All Bears are all Animals. False, because all bears are not all animals.

∀A∃BIsA(B, A) = A Bearis all animals. False, because a bear cannot be all animals.

∃A∀BIsA(B, A) = All Bears are a type of animal. True, because all bears are a type of animal.

∃A∃BIsA(B, A) = A Bear is an animal. True, because a bear is a type of animal.

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 3: Sets

Expected Duration: 1 week

Introduction

Many of the objects around you can be grouped into distinct objects of the same type. Example, if you look around, you will see different cars, which can be grouped together to form Set. Set is very basic mathematical concept use to group objects. It is basically used to show the relationship between each type of objects. The Venn diagram is always used to picture the set theory graphically. A set is a group that may contain none or one (1) or more elements. In formal methods, there are many ways to use set theory. One example will be to categorize many types of objects available in a system mostly in the form of data. Base on the purpose of the particular system or software the objects are properly grouped and then related to one another.

Learning Outcome for Study Unit 3 of Module 2.

At the end of this study unit 3, you should be able to:

1.1 Explain the meaning of important keywords that relate to Sets, which have been highlighted, (SAQ 1.1).
1.2 Identify and explain the different types of Sets, (SAQ 1.2).
1.3 Identify and explain the various Set operations, (SAQ 1.3).

1.1 Terminologies of Sets

Every area of life has important keywords or terminologies that relate to that area of life. There are important terminologies, which relate Set theory, they include the following: Elements of set, Cardinality, Membership, etc,

- **Elements:** The elements of a set are the various items that make a set. They are called the members of a set. Each member or element of a set must be unique.
- **Cardinality:** The cardinality of a set is the number of elements that make a set. Cardinality is denoted by vertical bars around the set. For example | {1, 3, 9, 15 } | = 4 | {1, 2, 3, . . .} | = ∞
- **Membership:** (∈) Membership happens when one element of a set is found inside another set. This symbol is normally used in describing a set for example A = {A ∈ Color of the rainbow} If A is a member of B, this is denoted A ∈ B. If A is not a member of B then A ∉ B A = {1, 2, 3, 4} therefore 4 ∈ A but 9 ∉ A B = {blue, white, red} therefore "blue" ∈ B but "pink" ∉ B

1.2 Different Types of Sets

Variety, they say is the spice of life. There are different types of Sets, which include the following: Universal set, Finite set, Infinite set, Subset, Superset, Disjoint sets, Null set, Family set, Power set, Equality of set etc. Each of these types of sets will be defined.

- **Universal Set:** This is the biggest set from which all other set is defined. All elements that is within the universe is considered necessary and elements not mentioned in the universe is considered none existence. Thus it is very important that we define the universe accurately.
- **Finite Set:** This is a set that contains finite cardinality. i.e. finite number of elements.
- **Infinite Set:** This is a set that contains infinite cardinality, i.e. infinite number of elements.
- **Subset:** The set A is a subset of a set B, A ⊆ B, if every element of set A is found in set B.
- **Superset:** The set B is a superset of A if the set B has every element of set A, (B ⊇ A).
- **Disjoint sets:** Two sets, A and B are disjoint sets if they do not contain any common element. Example, suppose A = {4, 2, 1, 3}, B = {5, 6, 7, 8}, sets A and B are called disjoint sets because theu do not have any common element.
- **Null set:** A null set is an empty set, i.e. a set that do not contain any element.
- **Family Set:** A set containing sets is called family of set. There are times when a set does not contain individual elements but it contains many subsets. Conveniently this is call a family set and is it describes using the curly bracket within a curly bracket. A = { {1, 2, 3, 4, 5} , {6, 7, 8, 9, 10} , {11, 12, 13, 14, 15} }
- **Power Set:** Remember that a set is a group that may contain none or one (1) or more elements. A power set means to show how many possible different ways to group all the elements in a set. In other words power set is the set of all subsets of a given set. A = {1, 2, 3} , A has 3 elements, there is 8 possible ways to arrange this 23 = 8. P(A) = { ∅ , {1}, {2}, {3}, {1,2}, {1,3}, {2,3}, {1, 2, 3} }
- **Equality of Set:** Two or more sets are equal if they have exactly the same number of elements with exactly the same value. Example, if A = {3, 4, 7, 8}, Z = {4, 3, 7, 8} therefore A = Z. If B = {3, 4, 7, 8, 4} and Y = {3, 4, 7, 8, 7}, therefore B = Y.

In-Text Question 1

What is the symbol for set membership?

1.3 Operations of Sets

Different objects that you use have different operations that you perform on them. Example, you drive a car, wash a car, but you do not wash a book; rather, you read a book. Sets have different operations that are performed on them. The various operations that are performed on set include the following: Union, Intersection, Difference, Complement etc. Each of these operations of set will be explained.

- **Union:** The Union of two sets, A and B is a set that combines all the elements of A and B. If A = {-1, 0, 1, 2, 5, 6, 9, 10} and B = {1, 2, 4, 6, 8, 10}, therefore, A ∪ B = {-1, 0, 1, 2, 4, 5, 6, 8, 9, 10}
- **Intersection:** The Intersection of two sets is the set that consists of common elements in the two sets. If A = {-1, 0, 1, 2, 5, 6, 9, 10} and B = {1, 2, 4, 6, 8, 10}, therefore, A ∩ B = {1, 2, 6, 10}
- **Difference:** The Difference of two sets, A and B, written as A-B is the set that you obtain after taking away all the elements of B from A. Also known as subtract, this shows only elements that is found in set A, but NOT found in set B. If A = {-1, 0, 1, 2, 5, 6, 9, 10} and B = {1, 2, 4, 6, 8, 10}, therefore, A - B = {-1, 0, 5, 9}
- **Complement:** The Complement of a set shows only elements that are not found in the set.

Answer to In-Text Question 1

∈

1.4 Summary of Study Unit 3

Having completed study unit 3, you have learnt that:

1. Set has important keywords/terminologies and their various definitions.
2. There are different types of Set.
3. There are different operations that are performed on Set.

1.5 Self Assessment Question for Study Unit 3 of Module 2

SAQ 1.1 (tests Learning Outcome 1.1)

Define the term Cardinality of a set.

SAQ 1.2 (Tests Learning Outcome 1.2)

Suppose set A = {1, 2}, find the Power set of A.

SAQ 1.3 (Tests Learning Outcome 1.3)

Assuming the following: U = {all the letters in the English alphabet} A = {a, e, i, o} B = {q, w, e, r, t, y, u, I, o, p}, find A ∩ B

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 4: Mathematical Proof

Expected Duration: 1 week

Introduction

Whatever you do, you should be able to convince others that what you are doing is the right thing. That is what Lawyers specialize in doing. Formal methods use mathematical proof to prove for the correctness of computer software. The unit introduces mathematical proof as a tool that can be used in formal methods.

Learning Outcome for Study Unit 4 of Module 2.

At the end of this study unit 4, you should be able to:

1.1 Explain the meaning of important keywords that relate to Mathematical proof, which have been highlighted, (SAQ 1.1).
1.2 Definition of mathematical proof, (SAQ 1.2).
1.3 Identify and explain the different typemathematical proofs, (SAQ 1.3).

1.1 Definition of Mathematical Proof

What you call a particular thing determines what that thing is to you. Mathematical proof simply means the use of mathematical language to be able to show that a statement is correct or true. No matter how the statement is twisted and turned or set against many different scenario, that statement comes up with the constant answer. Before a statement can be proved, it can have two parts, which are: the conditions followed by the result. For example, if it rain then I will be wet. This can be expressed in terms of proposition symbols as;

Rain → I am wet (if it rain then I will be wet).

The following are the basic terminologies that are related to mathematical proof.

- **Conjecture Hypothesis**
  This is a statement that is believe to be true but has yet to be proven.

- **Axiom/ Postulate**
  This is a statement that is taken for granted to be true even though it was never tested, but base on logic it is assumed to be true.
- **Paradox/ Antinomy**
  This is a statement which appears to contradict itself or contrary to expectations
- **Theorem**
  This is a statement that has been proven to be true.
- **Un-decidable**
  This is a statement that cannot be proven right or wrong.
- **Lemma**
  A proven theorem that is used to prove other statements
- **Converse**
  Theorem that is reversed or turned upside down or inward out thus a converse of a theorem need not be always true.

In-Text Question 1

What does Lemma means?

1.2 Different Types of Mathematical Proofs

Doing things in different ways, make it much interesting. There are different types of mathematical proof, and they consist of the following:

- **Direct Proof**
  In direct proof, the conclusion is established by logically combining the axioms, definitions, and earlier theorems. From the expression one can directly see the answer. For example: Rain → I am wet
- **Contradiction Proof**
  In proof by contradiction, if that statement is true and we logically contradict it then it will not be true anymore. For example: Rain → I am wet No Rain → I am Dry.
- **Contra-positive/Transposition Proof**
  Proof by transposition or contra-positive turns the statement inside out and upside down. This method swaps the result into the condition and negates both the result and condition.
  For example: Rain → I am wet
  I am Not wet → No Rain
- **Induction Proof**

This proof method insists that if the statement is true for one instance, it should be true for every instance.
For example: Rain → I am wet On Monday
(Rain → I am wet) On Tuesday
(Rain → I am wet) On Wednesday
(Rain → I am wet) On Thursday
(Rain → I am wet) On Friday
(Rain → I am wet)

Answer to In-Text Question 1

A proven theorem that is used to prove other statements

1.3 Summary of Study Unit 4

Having completed study unit 4, you have learnt that:

1. Mathematical proof has important keywords/terminologies and their various definitions.
2. Mathematical proof uses mathematical language to establish that a statement is true..
3. There are different types of mathematical proofs.

1.4 Self Assessment Question for Study Unit 4 of Module 2

SAQ 1.1 (Tests Learning Outcome 1.1)

Define the term Axiom/Postulate.

SAQ 1.2 (Tests Learning Outcome 1.2)

What does mathematical proof means.

SAQ 1.3 (Tests Learning Outcome 1.3)

List the different types of mathematical proofs.

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003

3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Notes for Self Assessment Questions (SAQs) for Study Unit 1.

SAQ 1.1

A Contingent is a proposition has at least one truth value that gives a true result and at least one false value that gives a true result

SAQ 1.2

First, use a truth table to evaluate the result of the proposition.

P	**Q**	**⌐ P**	**⌐Q**	**⌐ P ∧ ⌐Q**
T	T	F	F	F
T	F	F	T	F
F	T	T	F	F
F	F	T	T	T

What name can you use to describe this proposition, ⌐ P ∧ ⌐Q

SAQ 1.3 (Tests Learning Outcome 1.3)

P	Q	R	Q → R	.P→ (Q → R)
T	T	T	T	T
T	T	F	F	F
T	F	T	T	T
T	F	F	T	T
F	T	T	T	T
F	T	F	F	T
F	F	T	T	T
F	F	F	T	T

Notes for Self Assessment Questions (SAQs) for Study Unit 2.

SAQ 1.1

Quantifiers are part of the first order predicate logic that specify the size of constant/variable.

SAQ 1.2

- Variables/constants: Elements that change or do not change in the statement.
- A function that performs the variable: Action that is performed on the constant/variable.
- Result: This can be true or false
- Connectives/Operators: These are the various connectives/operators of first order predicate logic. They are the same as the operators of proposition.

SAQ 1.3

List and explain the two Quantifiers in first order predicate logic.

SAQ 1.4

1. Free variables are not applied to the quantifiers in first order predicate logic, while bound variables are applied to the quantifiers.

2. ∀A∀B IsA(B, A) = All Bears are all Animals. False, because all bears are not all animals.

   ∀A∃BIsA(B, A) = A Bear is all animals. False, because a bear cannot be all animals.

   ∃A∀BIsA(B, A) = All Bears are a type of animal. True, because all bears are a type of animal.

   ∃A∃BIsA(B, A) = A Bear is an animal. True, because a bear is a type of animal.

Notes for Self Assessment Questions (SAQs) for Study Unit 3.

SAQ 1.1

The cardinality of a set is the number of elements that make a set. Cardinality is denoted by vertical bars around the set. For example | {1, 3, 9, 15 } | = 4 | {1, 2, 3, . . .} | = ∞

.

SAQ 1.2

{ ∅ , {1}, {2}, {3}, {1,2} }

SAQ 1.3

{e, o}

Notes for Self Assessment Questions (SAQs) for Study Unit 4.

SAQ 1.1

This is a statement that is taken for granted to be true even though it was never tested, but base on logic it is assumed to be true.

SAQ 1.2

Mathematical proof simply means the use of mathematical language to be able to show that a statement is correct or true.

SAQ 1.3

- Direct Proof
- Contradiction Proof
- Contra-positive/Transposition Proof
- Induction Proof

Module 3: Z Specification Notation

Overview:

Sometimes, you need a special method for a special task. One of the classes or stages of formal methods is formal specification, which occur during the specification stage of the software. Using natural language to specify the software can lead to ambiguity, which eventually leads to error during the development of the software. Therefore, formal specification uses specification language to describe or specify the software to be developed. One of the specification language, which can be used to specify software is the Z, pronounced Zed specification language. This module introduces the use of Zed specification in specifying software. It consists of the following study units:

Study unit 1, Fundamentals of Z Notation,

Study unit 2, Steps in Using Z Notation to Specify Simple Systems.

Study unit 3, More Examples of the Use of Z Notation to Specify Systems

Study Unit 1: Fundamentals of Z Notations

Expected Duration: 1 week

Introduction

If you are a Lawyer, it means that there standard format that you use to handle cases in court. Formal specifications have different notations, which can be called specification languages. There are used to specific a software system. Z is one of the notations that can be used to specify a system. It uses first order predicate logic, set theory and other mathematical tools to specify a software system. This unit presents the fundamental concepts of the Z notation,

Learning Outcome for Study Unit 1

At the end of this study unit 1 of module 3, you should be able to:

1.1 Explain some of the important keywords that relates of the fundamentals of Z notation, (SAQ 1.1)
1.2 Identify with examples, the standard notations for different type of functions in Z, , (SAQ 1.2)

1.1 Terminologies of the Z Notation

In anything that you want to do, you must understand the fundamental terminologies about that thing. Before you can write create a formal specification in Z notation you must understand some basic concept and terminology. The following are some of the terminologies in the use of Z notation in specifying a system.

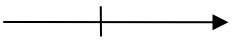
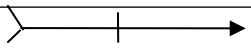
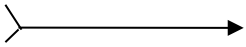
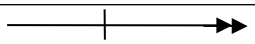
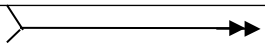
- **Data:**This refers to necessary data used in the system
- **Schema:** This refers to the structure of the Z notation, when specifying a particular function of the system. It consists of two sections, which are the signature and predicates.
- **Signature:** This is the (top section) of the Schema, which contains all the data and variables.
- **Predicate:** This is the (bottom section) of the Schema, which shows the relationship between the variables. The relationship must be true in every state of the system and is maintained by every operation on it.
- **Functions:** They are a special case of relations in which each element in one set (domain - dom) has one more value associated with another set (range - ran). They are represented by arrows.
- **Variable:** This refers a pre-condition or current condition, which means that variable name is in a state before any changes is made to it.
- **Variable':** This refers a post-condition, which means the variable' name is in a state after changes is made to it.
- **Variable?:** The value for this variable? comes from an input device. (input), which means that it is used to store input data.
- **Variable!:** The value for this variable! will be shown or listed out (output), which means that it is used to store output data.

In-Text Question 1

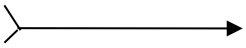
Define the term Schema?

1.2 Z Notations for Different Types of Functions

When you use different things to perform a task, there is need to distinguish one thing from the other. There are different types of functions, and they are distinguished from each other by the Z notation used for each and what the function does.

S/N	Function Name	Notation/Symbol	Description
1	Partial Function		X is a set and Y is a set X partial functions Y Each member X related to ONE (1) Y.
2	Total Function		X total functions Y These are partial functions whose domain is the whole of X; they relate each member of X to exactly one member of Y.
3	Partial Injection		This is a one-to-one mapping between elements in the domain and the range. The value in the domain may differ from the values in the range.
4	Total Injection		This is a one to many mapping where the domain completely populates the set.
5	Partial Surjections		This is a one to many mapping where the range completely populates the set Y.
6	Total Surjections		This is where both the domain and the range completely populate X and Y respectively.
7	Bijections		one-to-one total injective and surjective functions.

Answer to In-Text Question 1

Schema: This refers to the structure of the Z notation, when specifying a particular function of the system. It consists of two sections, which are the signature and predicates. 

1.3 Summary of Study Unit 1

Having completed study unit 1, you have learnt that:

1. Z notation has basic and fundamental keywords and the various definitions.
2. Each of the functions in Z has its own notations.

1.4 Self Assessment Question for Study Unit 1, of Module 3

SAQ 1.1 (Tests Learning Outcome 1.1)

Define the term Signature.

SAQ 1.2 (Tests Learning Outcome 1.2)

Write the Z notation for partial function.

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 2: Steps in Using Z Notation to Specify Simple System

Expected Duration: 1 week

Introduction

Everything you do requires well defined precise steps. Specifying a particular function of a system using Z notation also requires well define precise steps. This unit introduces the fundamental steps to be followed in specifying software function,

Learning Outcome for Study Unit 2

At the end of this study unit 2 of module 3, you should be able to:

1.1 Explain some of the important keywords that relates steps in specifying a system in Z, (SAQ 1.1)
1.2 Outline the steps for specifying a system in Z, with an example, (SAQ 1.2)

1.1 Steps in Specifying a System Using Z Notation

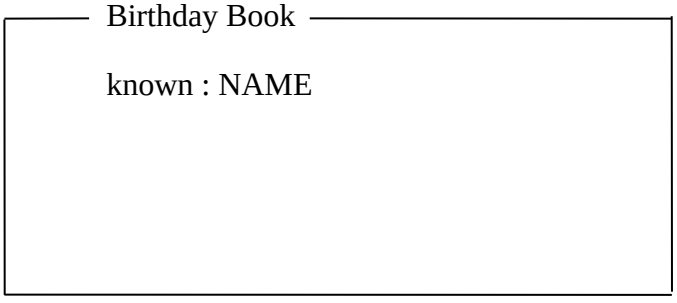
When you outline steps to be followed in performing a particular task, it helps in simplifying how to perform that task. The following steps can be used to specify software using the Z notation.

Step 1:

First you read the specification and list all the important data. As an illustration a small piece of software will be used. The software simply stores the names of people and their birthdays. In this Birthday book, there are two data, NAME and DATE. Every data is seen as a Mathematics SET.

Step 2:

Then we create an object schema and identify the state space.



birthday : NAME ⟶ DATE

known = dom birthday

Signature

In the above example, there is a variable called**known**, which represents set NAME(s). There is also a variable called **birthday**, which represents set DATE. This birthday (representing set DATE) has a function (partial function) that says every instance of known (representing set NAME) is related to ONLY ONE (1) instance inside set DATE. Meaning everyone has only 1 birthday.

Predicate

The **known** variable (set NAME) exists within a domain call Birthday.

The above Z specification can be rewritten using elements in the set as follows:

Birthday Book

known = {John, Mike, Susan}

birthday = {(John,25/3), (Mike, 22/12), (Susan, 22/12)}

known = dom birthday

The above Z specification can be rewritten as follows:

Birthday Book

known = {John, Mike, Susan}

birthday = {John ⟼ 25/3, Mike ⟼ 22/12, Susan ⟼ 22/12}

known = dom birthday

In-Text Question 1

Consider a small piece of software that stores information about students' regno, name and department. You are required to specify the system using Z notation.

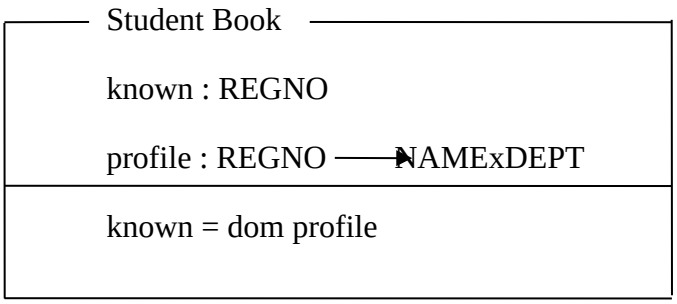
1.2 Summary of Study Unit 2

Having completed study unit 2, you have learnt that:

1. There are important keywords that relate to steps in using Z notation to specify a system.

2 The outlined steps can be used to specify a system using Z notation.

Answer to In-Text Question 1



1.3 Self Assessment Question for Study Unit 2, of Module 3

SAQ 1.1 (Tests Learning Outcome 1.1)

List the two parts of a Schema.

SAQ 1.2 (Tests Learning Outcome 1.2)

Consider a small piece of software that stores information about students' regno, name, dob and department. Assuming that you are to use Z notation to specify the system, identify the signature and the predicate.

References for Further Reading.

1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Study Unit 3: More Examples of the Use of Z Notation to Specify System.

Expected Duration: 1 week

Introduction

Anything that you want to learn, it is always easy to so by example, this is because learning by example is one of the most effective way you can learn. This units uses more examples to illustrate how to use Z notation to specify more functionalities of a system.

Learning Outcome for Study Unit 3

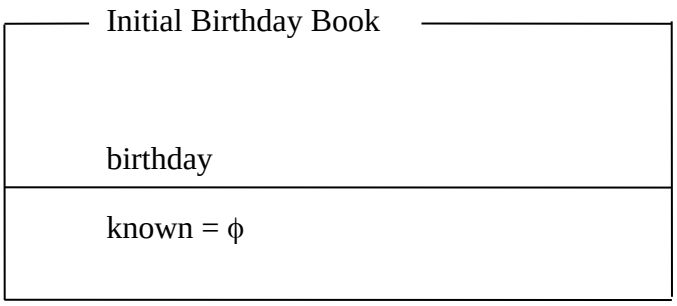
At the end of this study unit 3 of module 3, you should be able to:

1.1 Explain some of the important keywords that are related to examples of using Z notation to specify a system.
1.2 Use Z notation to specify the initial state of a simple Birthday Book system.
1.3 Use Z notation to specify the Add functionality, which adds a new birthday to the Birthday system.
1.4 Use the Z notation to specify the Edit operation, which modifies an existing birthday record.

1.1 Using Z Notation to Specify the Address Book App at its Initial State

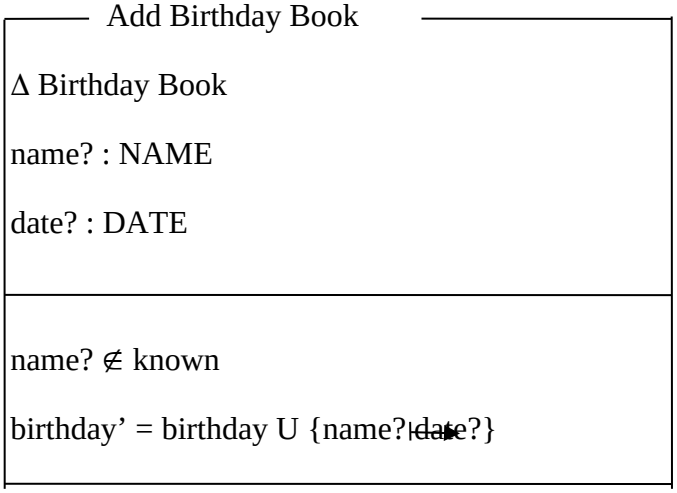
Every system that you have ever used begins with an initial or empty state, with no data on it. Consider the Birthday Book app in its initial state, without any data. The task is to use Z notation to specify the system at that initial state of no data.

Initially nothing exist expect the birthday book (something like the constructor in a class). We need an operation to set all default value to the variables. Take note that known is a default variable found in all schema.



1.2 Using Z Notation to Specify the Add Functionality to the Birthday System

You can make progress in life by adding new items into existing thing. The Birthday system can be populated with data by adding new birthday data. The Z specification for this functionality is show in the Schema below.



Signature

Remember previously that known has been declared to be a domain of birthday.

Δ means an operation schema for Birthday Book where the value in birthday domain will change. The variable called name? represents set NAME(s), which is an input by the user. The variable called date? represents set DATE, which will be input by the user.

Predicate

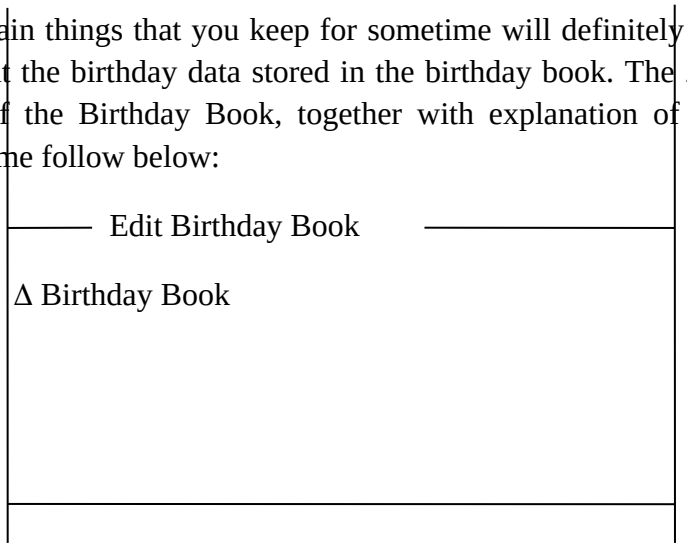
The name? must not be found in the known (domain of birthday)

In-Text Question 1

Show the Schema for the Z specification notation to specify the Delete operation on the Birthday Book. Explain the Signature and the Predicate of the Schema of the Z notation.

1.3 Using Z Notation to Specify the Edit Functionality of the Birthday System

Life is dynamic, certain things that you keep for sometime will definitely change with time. You may wish to edit the birthday data stored in the birthday book. The Z specification for the Edit operation of the Birthday Book, together with explanation of the signature and predicate of the Scheme follow below:



name? : NAME

date? : DATE

name? ∈ known

date! = birthday (name?)

date’ = date?

Signature

Remember previously that **known** has been declared to be a domain of birthday.

Δ means an operation schema for Birthday Book where the value in birthday domain will change. The variable called name? represents set NAME(s), which is input by the user. The variable called date? represents set DATE, which is also input by the user.

Predicate

The name? must be found in the known (domain of birthday) The date! will be shown base from the birthday domain where the name? is use as a parameter. The means that based othe given name,show the corresponding birthday. The date! will be updated with the new date? entered by the user.

Answer to In-Text Question 1

Delete Birthday Book

Δ Birthday Book

name? : NAME

date? : DATE

---

name? ∈ known

birthday’ = birthday - {name?date?}

Δ means an operation schema for Birthday Book where the value in birthday domain will change. The variable called name? represents set NAME(s), which is input by the user. The variable called date? represents set DATE, which will be input by the user.

Predicate

The name? must be found in the **known** (domain of birthday). The record with the name will be remove from the birthday domain.

1.4 Summary of Study Unit 3

Having completed study unit 3, you have learnt that:

1. There are important keywords that relate to examples of Z specifications (Schemas) of simple software system.
2. Z notation can be used to specify simple Birthday Book system.
3. Z notation can be used to specify the Add New operation/functionality of the simple Birthday Book system.
4. Z notation can be used to specify the Edit operation/functionality of the simple Birthday Book system.

1.5 Self Assessment Question for Study Unit 3, of Module 3

SAQ 1.1 (Tests Learning Outcome 1.1)

Distinguish between Signature and Predicate.

SAQ 1.2 (Tests Learning Outcome 1.2)

Use Z notation to show the Schema of the initial state of a simple students profile system, which stores information about student regno, name and department.

SAQ 1.3 (Tests Learning Outcome 1.3)

Use Z notation to show the Schema of the Add New functionality of a simple students profile system, which stores information about student regno, name and department.

SAQ 1.4 (Tests Learning Outcome 1.4)

Use Z notation to show the Schema of the Edit functionality of a simple students profile system, which stores information about student regno, name and department.

References for Further Reading.

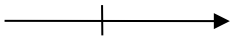
1. Formal Methods in Software Engineering, FTMS Global, Kuala Lumpur, Malaysia, 2011
2. Jonathan Bowen, Formal Specification and Documentation using Z: A Case Study Approach, 2003
3. Maximiliano Cristia, Introduction to the Z notation, Parma, India, 2020

Notes for Self Assessment Questions (SAQs) for Study Unit 1.

SAQ 1.1 (Tests Learning Outcome 1.1)

Signature is the (top section) of the Schema, which contains all the data and variables.

SAQ 1.2 (Tests Learning Outcome 1.2)



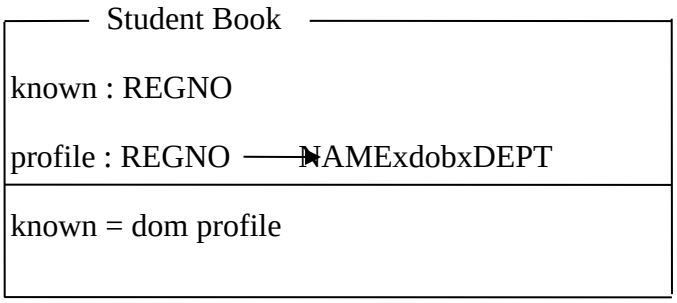
Notes for Self Assessment Questions (SAQs) for Study Unit 2.

SAQ 1.1 (Tests Learning Outcome 1.1)

Signature

Predicate

SAQ 1.2 (Tests Learning Outcome 1.2)

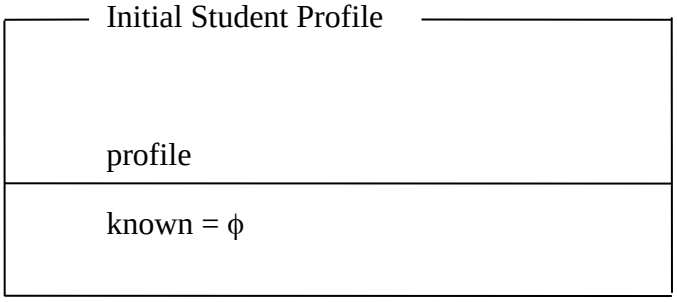


Notes for Self Assessment Questions (SAQs) for Study Unit 3.

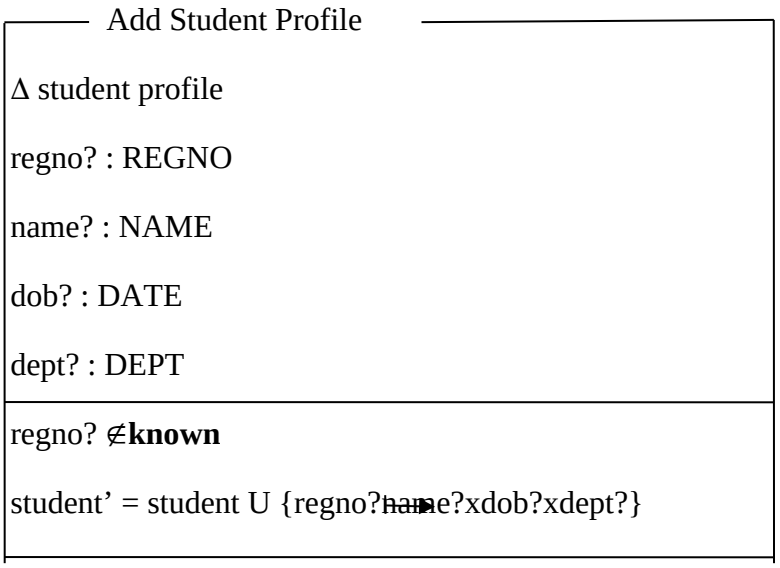
SAQ 1.1 (Tests Learning Outcome 1.1)

Signature is the upper part of Schema, while Predicate is the lower part of Schema.

SAQ 1.2 (Tests Learning Outcome 1.2)

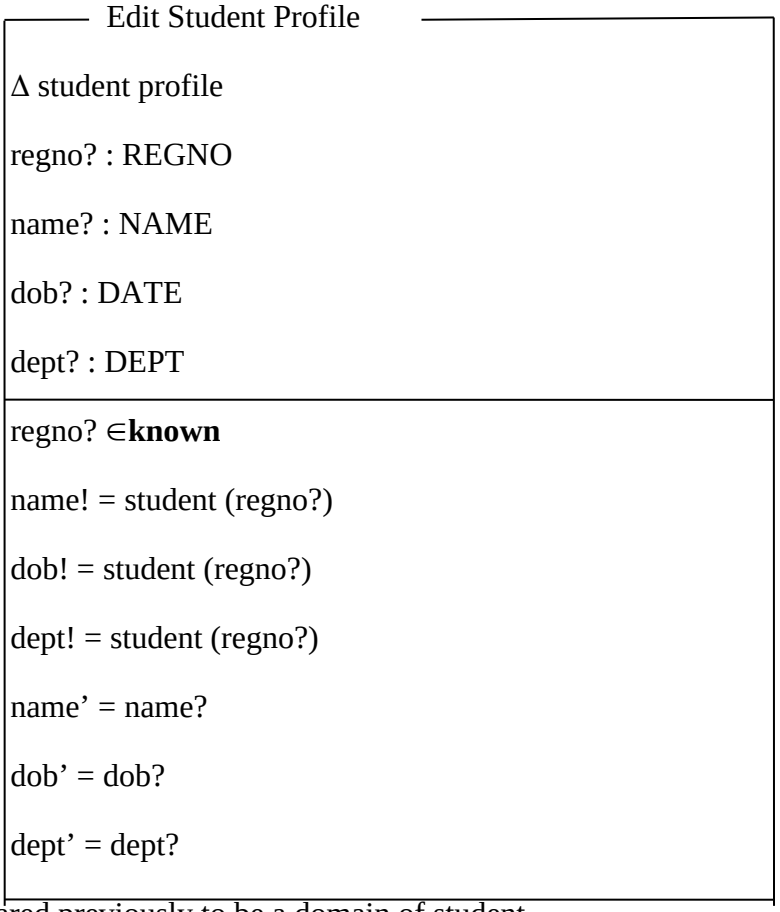


SAQ 1.3 (Tests Learning Outcome 1.3)



known has been declared to be a domain of **student**

SAQ 1.4 (Tests Learning Outcome 1.4)



known has been declared previously to be a domain of student